

MANUAL DEL PROGRAMADOR

Aplicación Móvil Interactiva 3D: Química RA

Universidad Técnica Particular de Loja (UTPL)

Entorno de Desarrollo: **Unity 6 (6000.3.6f1)**

Plataformas: **iOS / Android**

Base de Datos: **Firebase Firestore**

Fecha: **Julio 2026**

1. Introducción y Ficha Técnica

Este manual técnico está dirigido a programadores y desarrolladores responsables de mantener, actualizar o compilar el proyecto **AppQuímicaAR** (nombre comercial **App Química AR**). La aplicación es una herramienta didáctica e interactiva en 3D diseñada para dispositivos móviles con el fin de facilitar el aprendizaje de conceptos fisicoquímicos complejos como reacciones en superficies catalíticas de níquel, disociación de hidrógeno, decloración de dioxinas y barreras de energía.

Ficha Técnica de la Aplicación

Propiedad	Detalle
Nombre del Producto	App Química AR
Identificador de Paquete (Bundle ID)	com.DefaultCompany.QuimicaRA
Versión del Motor Unity	Unity 6 (6000.3.6f1)
Plataformas de Destino	Android (API nivel 29+) e iOS (iOS 12.0+)
Base de Datos / Backend	Firebase Firestore (Versión de SDK integrada en el proyecto)
Idiomas Soportados	Español
Modelador de Assets 3D	Blender (Modelos .blend exportados a .fbx)

2. Estructura y Arquitectura del Proyecto

El repositorio del proyecto sigue una estructura limpia, dividida entre recursos artísticos (Blender), configuraciones de compilación y código fuente de Unity.

Directorios Principales

```
Codigo-Fuente-App-Quimica/
├── Animaciones/                # Modelos y animaciones fuente en Blender (.blend)
│   ├── HIDROGENO.blend        # Modelo del Hidrógeno
│   ├── Niquel.blend           # Modelo del Níquel
│   ├── OCDD - copia.blend     # Modelo de la Dioxina OCDD
│   └── policlorodioxina/      # Assets adicionales de policlorodioxinas
├── AppQuimicaAR/              # Directorio del proyecto Unity
│   ├── Assets/                # Recursos de Unity
│   │   ├── Editor/            # Scripts exclusivos del Editor (compilación)
│   │   ├── Firebase/          # SDK y dependencias de Firebase
│   │   ├── Moléculas_Blend/    # Modelos FBX importados para uso en escenas
│   │   ├── Plugins/           # Plugins nativos para iOS/Android
│   │   ├── Prefabs/           # Elementos de UI prefabricados (ej. TermButton)
│   │   ├── Resources/         # Archivos de recursos dinámicos (BillingMode)
│   │   ├── Scenes/            # Escenas de Unity (.unity)
│   │   ├── Scripts/           # Código fuente C# (.cs)
│   │   └── TextMesh Pro/       # Paquete de texto TMPro para UI
│   ├── Packages/              # Gestor de paquetes de Unity (manifest.json)
│   └── ProjectSettings/        # Configuraciones globales de Unity
```

3. Flujo de Escenas y Persistencia de Estado

La aplicación cuenta con una serie de escenas consecutivas que guían al usuario a través del contenido teórico, las simulaciones interactivas de reacciones químicas y finalmente la evaluación de percepción.

Listado y Flujo de Escenas

Las escenas están indexadas en la configuración de compilación de Unity (`EditorBuildSettings.asset`) en el siguiente orden:

1. `PantallaInicial.unity`: Pantalla de bienvenida con las opciones principales (Iniciar simulación, Glosario, Cuestionario, Salir).
2. `Paso_alVacio1.unity`: Primera simulación fisicoquímica que muestra la superficie de Níquel y las moléculas de Hidrógeno en condiciones de vacío.
3. `Paso_alVacio2.unity`: Continuación del proceso de adsorción del hidrógeno sobre el catalizador de Níquel.
4. `Paso_alVacio2 1.unity`: Fase de transición y visualización de barreras energéticas adicionales en vacío.
5. `Paso_CumMetalico.unity`: Simulación avanzada que ilustra la interacción molecular sobre un cúmulo metálico.
6. `MenuPosAvance1.unity`: Pantalla intermedia que valida el progreso del usuario y le permite regresar o continuar.
7. `MenuPosAvance2.unity`: Segunda pantalla intermedia antes de la fase de evaluación final.
8. `PantallaCuestionario.unity`: Interfaz donde el usuario responde preguntas de percepción, cuyos datos se registran en Firebase Firestore.

Persistencia de Estado

La aplicación guarda el progreso de la escena del menú activo usando `PlayerPrefs` con la clave "CurrentMenu" mediante el script `MenuManager.cs`. Esto garantiza que, si la escena se recarga o se reabre la aplicación, el usuario retorne al último menú visitado.

4. Arquitectura de Código (Scripts C#)

El comportamiento lógico y la interactividad en 3D se definen mediante scripts C# organizados por funciones dentro de Assets/Scripts/. A continuación, se detalla cada componente:

4.1. Módulo del Cuestionario (CuestionarioManager.cs)

Esta clase gestiona la interfaz del cuestionario de evaluación del usuario. Genera de forma dinámica las preguntas, captura las respuestas en escala de estrellas (1 a 5) y controla los flujos de navegación en el cuestionario.

Estructura de Datos (secciones): Diccionario preconfigurado con cuatro categorías de preguntas:

9. **Aspectos técnicos:** Velocidad, interactividad, tipografía, formato.
10. **Innovación en el aula:** Interacción docente, atención, innovación.
11. **Metodología de enseñanza:** Aprendizaje, conceptos, participación, refuerzo.
12. **Actitudes:** Interés, masificación de herramientas, experiencia positiva.

- `PopulateSections()`: Instancia botones para cada sección de evaluación a partir del prefab `sectionButtonPrefab`.
- `OpenSection(string section)`: Inicializa la sección actual y despliega la ventana popup.
- `SelectRating(int rating)`: Guarda el puntaje asignado a la pregunta actual (1 a 5) y actualiza visualmente las estrellas.
- `NextQuestion()` / `PreviousQuestion()`: Controlan la navegación interna de preguntas de la sección activa.
- `CheckAllSectionsCompleted()`: Verifica si el usuario ha calificado todas las preguntas. Habilita el botón `finishButton`.
- `FinalizeQuiz()`: Llama a `FirebaseManager.Instance.SaveResponses` y redirige a la `PantallaInicial`.

4.2. Módulo de Base de Datos (FirebaseManager.cs)

Gestiona la conexión con Google Firebase y la escritura de datos en Firestore. Sigue el patrón Singleton para asegurar una única instancia durante la ejecución del juego.

Inicialización: Configura el SDK de Firebase a través del método asíncrono `CheckAndFixDependenciesAsync()`. Admite inicialización manual mediante `AppOptions` para evitar fallos de inicialización cruzada en compilaciones móviles:

```
AppOptions options = new AppOptions {
    ProjectId = "cuestionarios-7b7b2",
    DatabaseUrl = new System.Uri("https://cuestionarios-7b7b2-default-rtdb.firebaseio.com/"),
    MessageSenderId = "879493368500",
    StorageBucket = "cuestionarios-7b7b2.firebaseio.com",
    ApiKey = "[API_KEY_PLATAFORMA]",
    AppId = "[APP_ID_PLATAFORMA]"
}
```

```
};
```

- `SaveResponses(Dictionary<string, List<int>> respuestas, Action onComplete)`: Convierte el mapa de respuestas a un formato compatible (`Dictionary<string, object>`) y crea un documento con un identificador único (GUID) en la colección "Cuestionarios" de Firestore.

4.3. Módulo del Glosario (`GlossaryManager.cs`)

Carga dinámicamente un conjunto de conceptos fisicoquímicos en un menú de desplazamiento (Scroll View) para que el usuario pueda consultarlos.

Base de Datos Local: Diccionario en memoria con términos como Catálisis, Energía de activación, Energía de adsorción, Policlorodioxinas, etc.

- `PopulateGlossary()`: Recorre el diccionario e instancia botones de términos en la interfaz.
- `ShowDetail(string term)`: Despliega una ventana popup mostrando el término seleccionado junto a su definición teórica.

4.4. Módulo de Control de Energía (`LevelBarController.cs` y `LevelBarController1.cs`)

Asocia la posición de un control deslizante (Slider de UI) con los estados de animación de los átomos en 3D. Representa la energía de la reacción en electronvoltios (eV).

- Sigue el cambio de rotación de una molécula (`linkedObject.OnRotationChanged`).
- Ajusta el slider de energía de forma automática entre -1.5 eV y 2.26 eV.
- Al alcanzar el valor objetivo, dispara el trigger "Separate" en los componentes Animator de los átomos (`object1`, `object2` y `object3`) para simular la disociación.
- Deshabilita la interactividad del control deslizante al completarse el proceso.

4.5. Otros Controladores de Apoyo

- `RotateObject.cs`: Permite alternar la rotación de un objeto 3D de manera fluida usando una corrutina y `Quaternion.Slerp`. Dispara el evento `OnRotationChanged`.
- `AnimationController.cs`: Expone métodos públicos (`PlayAnimationsContinue`, `PlayAnimationsBack`) para desencadenar animaciones sincronizadas en un arreglo de animadores.
- `MenuFade.cs` y `MenuManager.cs`: Utilizan el componente `CanvasGroup` para realizar transiciones de opacidad (fade-in y fade-out) en las ventanas del menú principal.
- `Shownfo.cs` y `ShowInfoSC2.cs`: Desactivan los modelos 3D y los botones de control para dar prioridad a la visualización de paneles informativos específicos de la escena.

5. Integración con Firebase y Estructura de Datos

El almacenamiento de las respuestas de los cuestionarios se realiza en la base de datos no relacional Cloud Firestore.

Esquema del Documento en la Colección Cuestionarios

Cada documento generado representa una encuesta completada y tiene la siguiente estructura JSON:

```
{
  "Aspectos técnicos": [5, 4, 5, 5, 4],
  "Innovación en el aula": [5, 5, 4, 5, 5],
  "Metodología de enseñanza": [4, 5, 5, 4],
  "Actitudes": [5, 4, 5, 5]
}
```

NOTA

Cada elemento en las listas corresponde a la puntuación de estrellas asignada a cada pregunta en esa sección en particular (del 1 al 5).

Reglas de Seguridad Recomendadas para Firestore

Para permitir que los estudiantes guarden sus respuestas sin autenticación obligatoria (si el despliegue es meramente educativo), las reglas de Firestore de Firebase deben estar configuradas de la siguiente manera:

```
rules_version = '2';
service cloud.firestore {
  match /databases/{database}/documents {
    match /Cuestionarios/{document} {
      allow write: if true;
      allow read: if false; // Por privacidad, los usuarios no leen otras respuestas.
    }
  }
}
```

6. Proceso de Compilación (Build) y Despliegue

El proyecto cuenta con herramientas para automatizar el empaquetado de la aplicación en el directorio Builds/.

6.1. Compilación Automática de iOS

Se implementó el script `BuildScript.cs` dentro de la carpeta `Assets/Editor/` para automatizar la generación del proyecto Xcode en sistemas macOS.

13. Abrir el proyecto en Unity.
14. Hacer clic en el menú superior: Build -> Build iOS.
15. El script recopilará las escenas activas y exportará un proyecto de Xcode en la carpeta `Builds/iOS`.
16. Abrir el proyecto en Xcode, firmar el binario y compilar hacia un dispositivo físico de pruebas o App Store.

6.2. Compilación Manual para Android

Para generar el archivo ejecutable (.apk o .aab) para dispositivos Android:

17. Ir a File -> Build Settings en Unity.
18. Cambiar la plataforma de destino a **Android** y hacer clic en Switch Platform.
19. Asegurarse de que el SDK de Android esté correctamente vinculado en las preferencias de Unity.
20. Configurar la clave de firma digital en Project Settings -> Player -> Publishing Settings.
21. Presionar Build y guardar el archivo APK resultante.